

Design and Analysis of Algorithms

ASSIGNMENT 1

CO1: *Algorithm fundamentals, efficiency analysis, asymptotic analysis, mathematical analysis, recurrence analysis, Master Theorem, substitution, etc.*

Assignment Title: Design, Analyse and Validate an Efficient Search Strategy for a Large-Scale Student Record System (Total Marks - 50)

Duration: 1 week

Scenario — University Student Record Retrieval

A university maintains a large academic database containing student records, each identified by a unique Registration Number. Faculty and administrative staff frequently need to determine whether a particular student record exists and retrieve the corresponding information.

The database may contain:

1,000 → 10,000 → 100,000 → 1,000,000+ records

You are required to investigate, design and compare efficient algorithmic approaches for locating a given registration number as the database grows.

Your solution must remain practical for both small and very large datasets. Any preprocessing performed on the records must be included in your efficiency analysis.

Design Challenge:

The university now needs to process 50,000 search requests per hour on a database containing 1,000,000 student records. New student records are added only once per day.

Re-evaluate your original design and determine whether it should be retained, modified or redesigned.

Design and propose **at least three meaningfully different algorithmic strategies** for locating a given registration number.

The three approaches must represent **distinct algorithmic strategies**. Iterative and recursive implementations of the **same underlying algorithm will not be considered as two different approaches**.

Your proposed approaches must include:

- At least **one iterative approach**
- At least **one recursive approach**
- A clear explanation of the basic idea behind each approach
- Pseudocode or an appropriate flowchart for each approach
- A brief justification for selecting each approach for investigation
- Any preprocessing required by the approach, such as sorting or restructuring the data

You are expected to **design and select the approaches independently**. The appropriateness and diversity of the selected algorithmic strategies form part of the assessment.

Evidence Expected

Three meaningfully different algorithmic strategies, supported by pseudocode/flowcharts, identification of the design strategy, preprocessing requirements, and justification for selecting each approach.

For each of the proposed algorithms, identify the **basic operation** that predominantly determines its execution time.

Discuss:

- What the basic operation is
- Where it occurs in the algorithm
- Why it is appropriate to use this operation for efficiency analysis

- How the number of times it executes may change as the input size (n) increases

Do not simply name an operation. You are expected to **relate the selected operation to the behaviour of the algorithm**.

Evidence Expected

Correct identification and justification of the dominant/basic operation for each proposed algorithm.

TASK 3 – Derive the Time-Complexity Function Mathematically

6 Marks

Derive the time-complexity function for each proposed approach using mathematical analysis.

Discuss:

- The input size (n)
- The number of times the basic operation executes
- How the operation count changes with (n)
- The mathematical expression representing the running time
- Any preprocessing cost involved
- The overall cost when preprocessing and searching are considered together

You must show the **mathematical reasoning leading to the complexity result**.

Merely stating that an algorithm is $O(n)$, $O(\log n)$, etc., will not receive full credit.

Evidence Expected

A clear mathematical derivation of the time-complexity function for each approach, including relevant operation counts and preprocessing costs.

TASK 4 – Analyse Best, Average and Worst-Case Behaviour

4 Marks

Analyse the performance of each algorithm under different input conditions.

Discuss, wherever applicable:

- What input condition produces the **best case**

- What input condition produces the **average case**
- What input condition produces the **worst case**
- How the number of operations changes in each case
- Why these differences occur

Do not simply provide a table containing complexity values. You are expected to **explain the conditions that lead to each case**.

Evidence Expected

A justified best-, average- and worst-case analysis for the proposed approaches, supported by appropriate mathematical reasoning.

TASK 5 – Perform Asymptotic Analysis

3 Marks

Express and compare the growth behaviour of your proposed algorithms using appropriate asymptotic notation.

Discuss:

- Appropriate use of O , Ω and/or Θ , wherever relevant
- How each algorithm behaves as n becomes very large
- How the growth rates of the proposed approaches differ
- What the growth behaviour means when the dataset increases from **1,000 to 1,000,000+ records**

You are expected to **interpret the asymptotic behaviour in the context of the given application**, rather than merely state the notation.

Evidence Expected

Correct asymptotic representation and a meaningful comparison of the growth behaviour of the proposed approaches.

TASK 6 – Formulate the Recurrence Relation

3 Marks

For the recursive approach, formulate a recurrence relation directly from the behaviour of the algorithm.

Clearly identify:

- The input size of the original problem
- The size of the recursive subproblem
- The number of recursive calls
- The work performed outside the recursive call
- The base case
- How these components lead to the recurrence relation

The recurrence must be **derived from your algorithm**, rather than simply writing a known recurrence formula.

Evidence Expected

A correctly formulated recurrence relation with clear identification of its components and base case.

TASK 7 – Solve the Recurrence Relation

4 Marks

Solve the recurrence relation obtained in Task 6 using an appropriate analytical method.

You may use:

- Substitution method
- Iteration method
- Master Theorem
- Another appropriate analytical technique

Discuss:

- Why the selected method is applicable
- The steps involved in solving the recurrence
- The resulting complexity
- What the result indicates about the behaviour of the recursive algorithm as (n) increases

You are expected to **show the analytical process**, rather than provide only the final complexity.

Evidence Expected

A complete recurrence solution using an appropriate method, together with justification of the method and interpretation of the result.

TASK 8 – Implement and Test the Algorithms

4 Marks

Implement all the proposed algorithmic approaches and experimentally test their behaviour.

Use datasets of approximately:

1,000 → 10,000 → 100,000 → 1,000,000 records

Ensure that:

- All algorithms produce the correct result
- The same or equivalent datasets are used for comparison
- Testing conditions are kept reasonably consistent
- Appropriate test cases are included
- The implementation corresponds to the algorithms analysed in the previous tasks

The purpose of implementation is **not merely to demonstrate working code**, but to generate evidence for validating your theoretical analysis.

Evidence Expected

Working implementations of the proposed algorithms together with a clearly described and reproducible testing procedure.

TASK 9 – Record Experimental Performance

3 Marks

Collect and present experimental performance data for each algorithm across the different input sizes.

Record appropriate measures such as:

- Execution time
- Number of comparisons
- Number of key operations
- Input size
- Other relevant performance measures, where justified

Present the results using appropriate:

- Tables
- Graphs
- Performance plots

Ensure that the presentation allows the different approaches to be easily compared.

Evidence Expected

Systematically collected experimental data presented clearly using appropriate tables and/or graphs.

TASK 10 – Compare Theory with Experimental Behaviour

4 Marks

Compare your theoretical analysis with your experimental results.

Discuss:

- Whether the observed growth agrees with the theoretical complexity
- Which algorithm scales better as (n) increases
- Any differences between theoretical prediction and actual execution
- Possible reasons for the differences

You are expected to **interpret the results**, rather than simply reproduce tables or graphs.

Evidence Expected

A reasoned comparison between theoretical and experimentally observed performance.

TASK 11 – Recommend and Defend the Final Design

4 Marks

Based on your mathematical analysis and experimental findings, recommend the most appropriate algorithmic solution for the university student-record system.

Discuss:

- Which approach you recommend
- Why it is more appropriate than the alternatives
- Its theoretical efficiency
- Its experimentally observed performance
- Any preprocessing cost involved

- Relevant trade-offs or limitations
- Whether the solution remains practical as the database grows

Your recommendation must be based on **evidence obtained from your own analysis and experimentation**.

Simply identifying the algorithm with the lowest Big-O complexity is not sufficient.

Evidence Expected

A technically justified final design recommendation supported by both mathematical analysis and experimental evidence.

DAY-4 DESIGN CHALLENGE

TASK 12 – Re-evaluate and Adapt the Design

8 Marks

Changed Requirement

The university now needs to process **50,000 search requests per hour** on a database containing **1,000,000 student records**.

However, new student records are added only **once per day**.

Re-evaluate your original design in response to these changed operating conditions.

You may:

RETAIN → MODIFY → or REDESIGN

your original solution.

A. Impact Analysis – 2 Marks

Discuss:

- What has changed from the original problem
- Which original assumptions are affected
- Why the frequency of searches is now important
- Why the relatively low frequency of record updates may influence the design

Evidence Expected

A clear analysis of how the changed requirements affect the original algorithmic design.

B. Evaluation of the Original Design – 2 Marks

Evaluate whether your original recommended solution remains appropriate.

Discuss:

- Its expected behaviour under 50,000 searches per hour
- Whether its preprocessing cost is acceptable under the new conditions
- Whether another approach could provide better overall performance

Evidence Expected

A technically supported evaluation of the suitability of the original design under the new operating conditions.

C. Adaptation or Redesign – 2 Marks

Decide whether to:

Retain, modify or redesign the original solution.

If you modify or redesign it:

- Describe the revised algorithmic strategy
- Explain what has changed
- Determine the effect on computational complexity

If you retain the original solution, provide sufficient technical justification for why no redesign is necessary.

Evidence Expected

An appropriate design response to the changed requirement, with clear algorithmic reasoning.

D. Evidence-Based Defence – 2 Marks

Defend your final design decision using evidence.

Use:

- Mathematical complexity analysis

- Experimental results, where appropriate
- Search frequency
- Preprocessing cost
- Dataset size
- Relevant computational trade-offs

Your final decision must follow logically from your analysis.

Evidence Expected

A well-supported final design decision integrating theoretical analysis, experimental evidence and the changed system constraints.

MARK DISTRIBUTION

Task	Marks
Task 1 – Algorithm Design	5
Task 2 – Basic Operation	2
Task 3 – Mathematical Complexity Analysis	6
Task 4 – Best/Average/Worst-Case Analysis	4
Task 5 – Asymptotic Analysis	3
Task 6 – Recurrence Formulation	3
Task 7 – Recurrence Solution	4
Task 8 – Implementation & Testing	4
Task 9 – Experimental Performance	3
Task 10 – Theory vs Experiment	4
Task 11 – Final Design Recommendation	4
Subtotal	42
Task 12 – Day-4 Design Challenge	8
TOTAL	50

Sure. I'll give it **normally**, with proper headings, numbering, equations, tables, and pseudocode so you can simply select and copy it into Word. No code block.

DESIGN AND ANALYSIS OF ALGORITHMS – ASSIGNMENT 1

Assignment Title

Design, Analyse and Validate an Efficient Search Strategy for a Large-Scale Student Record System

CO1

Algorithm fundamentals, efficiency analysis, asymptotic analysis, mathematical analysis, recurrence analysis, Master Theorem, substitution, etc.

Introduction

A university maintains a large academic database containing student records. Each student record is uniquely identified by a Registration Number. Faculty members and administrative staff frequently need to determine whether a particular student record exists and retrieve the corresponding information.

The database may contain approximately 1,000, 10,000, 100,000, and 1,000,000 or more records. Therefore, an efficient searching strategy is required that remains practical as the size of the database increases. Any preprocessing required by an algorithm, such as sorting or constructing a data structure, must also be considered when analysing its efficiency. Design and Analysis of Algorithm...

For this assignment, three different searching strategies are designed and analysed:

1. Linear Search
2. Recursive Binary Search
3. Hash Table Search

These algorithms represent different algorithmic strategies: brute force, divide and conquer, and hashing.

TASK 1 – DESIGN THE ALGORITHM

5 Marks

The assignment requires three meaningfully different algorithmic strategies, including at least one iterative and at least one recursive approach. Each approach must include its basic idea, pseudocode, preprocessing requirements, and justification. Design and Analysis of Algorith...

1. Linear Search

Design Strategy

Brute Force / Sequential Search

Basic Idea

Linear Search examines the registration numbers one by one from the beginning of the database until the required registration number is found.

It does not require the data to be sorted.

For example, if the registration numbers are:

1001, 1025, 1050, 1075, 1100

and the required registration number is 1075, the algorithm performs:

1001 → Not found

1025 → Not found

1050 → Not found

1075 → Found

Thus, the algorithm stops when the required record is located.

Pseudocode

LINEAR_SEARCH(A, n, key)

1. Start
2. Set $i = 0$
3. Repeat while $i < n$
4. If $A[i] = \text{key}$, return i
5. Otherwise increment i
6. If all records are checked, return -1
7. Stop

Preprocessing

No preprocessing is required.

Preprocessing cost = 0.

Justification

Linear Search is selected because it is simple, easy to implement, and works on both sorted and unsorted data. It provides a useful baseline for comparing more advanced searching algorithms.

However, when the database becomes very large, the algorithm may have to inspect a large number of records.

2. Recursive Binary Search

Design Strategy

Divide and Conquer

Basic Idea

Binary Search works only on sorted data.

Instead of checking every record, it checks the middle element of the current search range.

If the required registration number is smaller than the middle element, the right half is discarded.

If the required registration number is larger than the middle element, the left half is discarded.

Therefore, the search space is reduced approximately as follows:

$$n \rightarrow n/2 \rightarrow n/4 \rightarrow n/8 \rightarrow \dots \rightarrow 1$$

Pseudocode

BINARY_SEARCH(A, low, high, key)

1. If $low > high$, return -1.
2. Calculate:

$mid = low + (high - low) / 2$

3. If $A[mid] = key$, return mid.
4. If $key < A[mid]$, recursively search the left half.
5. Otherwise, recursively search the right half.
6. Stop when the key is found or the search range becomes empty.

Preprocessing

Binary Search requires sorted data.

Therefore, if the original database is unsorted, sorting must be performed first.

For an efficient sorting algorithm:

Sorting cost = $O(n \log n)$

After sorting:

Search cost = $O(\log n)$

Justification

Binary Search is selected because it demonstrates the Divide-and-Conquer strategy and provides the required recursive approach. It is much more efficient than Linear Search for large sorted datasets.

3. Hash Table Search

Design Strategy

Hashing / Direct Access

Basic Idea

Hashing uses a hash function to calculate the location of a registration number in a hash table.

A simple hash function is:

$$h(k) = k \bmod m$$

where:

k = registration number

m = size of hash table

For example:

If $k = 12345$ and $m = 100$,

$h(12345) = 12345 \bmod 100$

$h(12345) = 45$

Therefore, the registration number is associated with position 45.

Pseudocode

BUILD_HASH_TABLE(A, n)

1. Create an empty hash table H .
2. For every registration number in A :
3. Calculate $\text{index} = \text{hash}(\text{key})$.
4. Insert the record into $H[\text{index}]$.
5. Return H .

HASH_SEARCH(H, key)

1. Calculate $\text{index} = \text{hash}(\text{key})$.
2. Access $H[\text{index}]$.
3. Search the corresponding bucket.
4. If the key is found, return the record.
5. Otherwise return -1.

Collision Handling

Sometimes two different registration numbers can produce the same hash value. This is called a collision.

Collision handling can be performed using:

- Separate chaining
- Open addressing

For this design, separate chaining can be used.

Preprocessing

The hash table must be constructed before searching.

Expected construction cost:

$O(n)$

Expected search cost under good hashing:

$O(1)$

Justification

Hash Search is selected because it provides a fundamentally different searching strategy. It is especially useful when a large number of search operations are performed on a relatively stable database.

Comparison of the Three Algorithms

Feature	Linear Search	Binary Search	Hash Search
Strategy	Brute Force	Divide and Conquer	Hashing
Type	Iterative	Recursive	Hash-table based
Data requirement	Any order	Sorted	Hash table
Preprocessing	None	Sorting	Hash-table construction
Search complexity	$O(n)$	$O(\log n)$	Expected $O(1)$
Extra memory	Very low	Recursion stack	Hash table
Main advantage	Simple	Fast for sorted data	Very fast expected lookup
Main disadvantage	Slow for large n	Sorting required	Extra memory and collisions

Thus, the three algorithms provide three substantially different approaches to the student record retrieval problem.

TASK 2 – IDENTIFY THE BASIC OPERATION

2 Marks

The basic operation is the operation that is performed repeatedly and has the greatest influence on the running time. The rubric requires not only identifying the operation

but also explaining where it occurs and how its frequency changes with input size.
Design and Analysis of Algorith...

2.1 Linear Search

The basic operation is the comparison:

$$A[i] = \text{key}$$

It occurs inside the loop.

For n records, the number of comparisons can range from 1 to n .

As n increases, the number of comparisons increases directly.

Therefore, the dominant operation is the key comparison.

2.2 Binary Search

The basic operation is:

$$A[\text{mid}] = \text{key}$$

This comparison occurs at every recursive call.

After each comparison, approximately half of the remaining records are eliminated.

Therefore, the number of comparisons grows as:

$$\log_2 n$$

Thus, the middle-element comparison determines the running time.

2.3 Hash Search

The basic operations are:

1. Computing the hash value.
2. Accessing the corresponding bucket.
3. Comparing the key with entries in that bucket.

With a good hash function, only a small number of entries need to be checked.

Therefore, the expected search operation is approximately constant.

TASK 3 – DERIVE THE TIME-COMPLEXITY FUNCTION MATHEMATICALLY

6 Marks

The rubric requires mathematical reasoning, operation counts, input size, preprocessing cost, and the combined preprocessing and searching cost. Design and Analysis of Algorith...

3.1 Linear Search

Let:

n = number of student records.

The basic operation is the comparison:

$A[i] = \text{key}$

Best Case

The key is located at the first position.

Therefore:

$$C_{\text{best}}(n) = 1$$

Hence:

$$T_{\text{best}}(n) = c$$

Therefore:

$$\Theta(1)$$

Worst Case

The key is at the last position or does not exist.

Therefore:

$$C_{\text{worst}}(n) = n$$

The running time can be written as:

$$T_worst(n) = cn + c_0$$

Ignoring constants:

$$T_worst(n) = \Theta(n)$$

Average Case

Assuming the key is equally likely to occur at any position:

$$C_average(n) = (1 + 2 + 3 + \dots + n) / n$$

Using:

$$1 + 2 + \dots + n = n(n + 1)/2$$

Therefore:

$$C_average(n)$$

$$= [n(n + 1)/2] / n$$

$$= (n + 1)/2$$

Thus:

$$T_average(n) = c(n + 1)/2 + c_0$$

Therefore:

$$T_average(n) = \Theta(n)$$

Overall

Linear Search:

$$\text{Best case} = \Theta(1)$$

$$\text{Average case} = \Theta(n)$$

$$\text{Worst case} = \Theta(n)$$

No preprocessing is required.

3.2 Binary Search

For Binary Search, the input size is n .

After every comparison, the problem size is approximately divided by 2.

Therefore:

$$n \rightarrow n/2 \rightarrow n/4 \rightarrow n/8 \rightarrow \dots$$

After k steps:

$$n / 2^k$$

The algorithm terminates when the remaining problem contains one element.

Therefore:

$$n / 2^k = 1$$

Hence:

$$n = 2^k$$

Taking logarithm on both sides:

$$\log_2 n = k$$

Therefore:

$$k = \log_2 n$$

Each recursive level performs constant work.

Thus:

$$T(n) = c \log_2 n + c_0$$

Therefore:

$$T(n) = \Theta(\log n)$$

Including Sorting

If the original records are unsorted, sorting is required.

Sorting:

$$T_{\text{sort}}(n) = \Theta(n \log n)$$

For one search:

$$T_{\text{total}}(n) = \Theta(n \log n) + \Theta(\log n)$$

Therefore:

$$T_{\text{total}}(n) = \Theta(n \log n)$$

However, if the sorted data is reused for m searches:

$$\begin{aligned} T_{\text{total}}(n,m) \\ = \Theta(n \log n) + m\Theta(\log n) \end{aligned}$$

Therefore:

$$\begin{aligned} T_{\text{total}}(n,m) \\ = \Theta(n \log n + m \log n) \end{aligned}$$

This distinction is important because the preprocessing cost is paid once while the sorted data can be reused for many searches.

3.3 Hash Search

Let n be the number of records.

The hash table must first be constructed.

Each record is inserted into the table.

Therefore:

$$T_{\text{build}}(n) = c_1 n$$

Hence:

$$T_{\text{build}}(n) = \Theta(n)$$

For an individual search, the hash function is calculated and the corresponding bucket is accessed.

Expected number of operations is approximately constant.

Therefore:

$$T_{\text{search}}(n) = \Theta(1)$$

For m searches:

$$T_{\text{total}}(n,m)$$

$$= \Theta(n) + m\Theta(1)$$

Therefore:

$$T_{\text{total}}(n,m)$$

$$= \Theta(n + m)$$

Thus, hashing has a one-time construction cost but provides very fast expected search operations.

=====

TASK 4 – BEST, AVERAGE AND WORST-CASE BEHAVIOUR

The rubric requires explanation of the input condition producing each case and why the number of operations changes. Design and Analysis of Algorith...

4.1 Linear Search

Best Case

The required registration number is the first record.

Comparisons = 1

Complexity:

$$\Theta(1)$$

Average Case

The registration number is equally likely to occur at any position.

Average comparisons:

$$(n + 1)/2$$

Complexity:

$$\Theta(n)$$

Worst Case

The registration number is at the last position or does not exist.

Comparisons = n

Complexity:

$$\Theta(n)$$

4.2 Binary Search

Best Case

The required registration number is exactly the middle element.

Comparisons = 1

Complexity:

$$\Theta(1)$$

Average Case

The record is found after several divisions of the search space.

Number of comparisons is proportional to:

$$\log_2 n$$

Therefore:

$$\Theta(\log n)$$

Worst Case

The record is located near the deepest level or does not exist.

Maximum number of divisions is approximately:

$$\lceil \log_2 n \rceil + 1$$

Therefore:

$$\Theta(\log n)$$

4.3 Hash Search

Best Case

The key is found immediately in the calculated bucket.

Complexity:

$$\Theta(1)$$

Average Case

With a good hash function and low collision rate, only a small number of records are examined.

Expected complexity:

$$\Theta(1)$$

Worst Case

If many records collide into the same bucket, the bucket may contain many records.

In the extreme case, all n records may be in one bucket.

Therefore:

Worst-case complexity:

$$\Theta(n)$$

This shows that Hash Search is expected $O(1)$, but its worst case can become $O(n)$.

=====

TASK 5 – ASYMPTOTIC ANALYSIS

Asymptotic analysis describes how the running time grows as the input size becomes very large. The rubric requires O , Ω and/or Θ and interpretation for datasets growing to 1,000,000+ records. Design and Analysis of Algorith...

Linear Search

Best case:

$$\Omega(1)$$

Worst case:

$O(n)$

Average:

$\Theta(n)$

Therefore, the overall growth is linear.

If the number of records increases by 100 times, the worst-case search work can also increase by approximately 100 times.

Binary Search

Best case:

$\Omega(1)$

Worst case:

$O(\log n)$

Average:

$\Theta(\log n)$

The search space is divided by two repeatedly.

For 1,000,000 records:

$\log_2(1,000,000) \approx 19.93$

Therefore, approximately 20 levels are sufficient to identify a location.

This is dramatically smaller than one million sequential comparisons.

Hash Search

Expected:

$\Theta(1)$

Worst case:

$O(n)$

A good hash function provides approximately constant expected search time.

Therefore, increasing the number of records does not proportionally increase the expected number of search operations.

Growth Comparison

For very large n :

n grows much faster than $\log n$.

Therefore:

$O(n) > O(\log n)$

Expected $O(1)$ grows even more slowly.

The theoretical ranking is:

$O(1) < O(\log n) < O(n)$

For a database containing 1,000,000+ records, Linear Search may require a very large number of comparisons, whereas Binary Search requires only around 20 levels, and Hash Search can provide approximately constant expected lookup time.

=====

TASK 6 – FORMULATE THE RECURRENCE RELATION

The recursive algorithm selected is Recursive Binary Search.

The recurrence must be derived directly from the behaviour of the algorithm. Design and Analysis of Algorith...

Let $T(n)$ represent the running time for searching n records.

At each recursive call:

1. One middle element is examined.
2. Only one half of the data is searched recursively.
3. The other half is discarded.

Therefore:

Recursive subproblem size = $n/2$

Number of recursive calls = 1

Work outside the recursive call = constant c .

Therefore:

$$T(n) = T(n/2) + c$$

Base Case

When only one element remains:

$$T(1) = c_0$$

Therefore, the recurrence is:

$$T(n) = T(n/2) + c, n > 1$$

$$T(1) = c_0$$

=====

TASK 7 – SOLVE THE RECURRENCE RELATION

We use the substitution/iteration method because the recurrence was directly obtained from the recursive Binary Search algorithm.

Given:

$$T(n) = T(n/2) + c$$

Base case:

$$T(1) = c_0$$

Step 1

Expand $T(n/2)$:

$$T(n/2) = T(n/4) + c$$

Therefore:

$$T(n)$$

$$= T(n/4) + c + c$$

$$T(n)$$

$$= T(n/4) + 2c$$

Step 2

Expand again:

$$T(n/4)$$

$$= T(n/8) + c$$

Therefore:

$$T(n)$$

$$= T(n/8) + 3c$$

Step 3

After k expansions:

$$T(n)$$

$$= T(n/2^k) + kc$$

The recursion stops when:

$$n/2^k = 1$$

Therefore:

$$n = 2^k$$

Taking logarithm:

$$k = \log_2 n$$

Substituting:

$$T(n)$$

$$= T(1) + c \log_2 n$$

Using:

$$T(1) = c_0$$

Therefore:

$$T(n)$$

$$= c_0 + c \log_2 n$$

Ignoring constants:

$$T(n) = \Theta(\log n)$$

Therefore:

BOXED RESULT:

$$T(n) = \Theta(\log n)$$

Interpretation

The logarithmic result means that Binary Search becomes extremely efficient as the database becomes large.

Doubling the database does not double the number of search operations. Instead, it increases the number of levels by only approximately one.

For example:

$$n = 1,000,000$$

$$\log_2(1,000,000) \approx 20$$

Thus, only about 20 levels of searching are required.

=====

TASK 8 – IMPLEMENT AND TEST THE ALGORITHMS

The assignment requires all proposed approaches to be implemented and tested using approximately:

$$1,000 \rightarrow 10,000 \rightarrow 100,000 \rightarrow 1,000,000 \text{ records.}$$

The same or equivalent datasets and reasonably consistent testing conditions should be used. Design and Analysis of Algorithm...

Experimental Setup

The following dataset sizes are used:

1. 1,000 records
2. 10,000 records
3. 100,000 records
4. 1,000,000 records

Each record contains a unique registration number.

The same target values should be used as far as possible for comparing the algorithms.

Test Cases

For each dataset, the following cases can be tested:

1. Key at first position
2. Key near middle
3. Key at last position
4. Key not present

Linear Search Implementation

LINEAR_SEARCH(A, n, key)

```
for i = 0 to n-1
    if A[i] == key
        return i
return -1
```

Binary Search Implementation

BINARY_SEARCH(A, low, high, key)

```
if low > high
    return -1
```

```
mid = low + (high-low)/2
```

```
if A[mid] == key
    return mid
```

```
if key < A[mid]
    return BINARY_SEARCH(A, low, mid-1, key)
```

```
return BINARY_SEARCH(A, mid+1, high, key)
```

Hash Search Implementation

HASH_SEARCH(H, key)

```
index = hash(key)
```

```
search H[index]
```

```
if key is present  
    return record
```

```
return -1
```

Testing Procedure

1. Generate the required number of student records.
2. Create equivalent datasets for all algorithms.
3. Sort the data for Binary Search.
4. Build the hash table for Hash Search.
5. Select search keys.
6. Execute each algorithm.
7. Record execution time.
8. Count key comparisons where appropriate.
9. Repeat the experiment if required to reduce measurement variation.
10. Compare the results.

Correctness must first be verified before comparing performance.

=====

TASK 9 – RECORD EXPERIMENTAL PERFORMANCE

The rubric requires experimental performance data such as execution time, comparisons and input size, presented using tables or graphs. Design and Analysis of Algorith...

The following table can be used to present the experimental results.

Number of Records	Linear Search	Binary Search	Hash Search
1,000	Record actual time	Record actual time	Record actual time
10,000	Record actual time	Record actual time	Record actual time
100,000	Record actual time	Record actual time	Record actual time
1,000,000	Record actual time	Record actual time	Record actual time

A comparison-count table can also be used:

Input Size	Linear Search Comparisons	Binary Search Comparisons	Hash Search Comparisons
1,000	Record value	Record value	Record value
10,000	Record value	Record value	Record value
100,000	Record value	Record value	Record value
1,000,000	Record value	Record value	Record value

IMPORTANT:

Actual execution times must be measured from the implementation rather than invented. The table above is the correct format for recording the experimental results.

Expected Trend

The expected theoretical trend is:

Linear Search:

Number of comparisons increases approximately linearly.

Binary Search:

Number of comparisons increases logarithmically.

Hash Search:

Expected number of comparisons remains approximately constant when the hash function performs well.

=====

TASK 10 – COMPARE THEORY WITH EXPERIMENTAL BEHAVIOUR

The theoretical analysis predicts the following:

Linear Search:

$\Theta(n)$

Binary Search:

$\Theta(\log n)$

Hash Search:

Expected $\Theta(1)$

As the dataset increases from 1,000 to 1,000,000 records, Linear Search should show the largest increase in searching work because its running time grows directly with n .

Binary Search should show a much smaller increase because its running time grows logarithmically.

Hash Search should show approximately constant expected search performance, assuming a good hash function and low collision rate.

Why Actual Results May Differ

The measured execution time may not exactly match theoretical complexity because of:

1. Processor speed
2. Memory hierarchy
3. Cache effects
4. Programming language overhead
5. Operating-system scheduling
6. Garbage collection
7. Hash collisions
8. Data distribution
9. Sorting time
10. Measurement overhead

Big-O analysis describes growth behaviour rather than exact execution time.

Therefore, small differences in measured values do not necessarily contradict the theoretical analysis.

Overall Comparison

If the experimental results show that Linear Search becomes increasingly slower as n increases, the experiment supports $\Theta(n)$.

If Binary Search remains fast even when n increases significantly, the result supports $\Theta(\log n)$.

If Hash Search maintains nearly constant expected lookup time, the result supports expected $\Theta(1)$.

=====

TASK 11 – RECOMMEND AND DEFEND THE FINAL DESIGN

The final design must be selected using both mathematical analysis and experimental

evidence rather than simply selecting the algorithm with the smallest Big-O notation.
Design and Analysis of Algorith...

Recommended Algorithm

For the university student-record system, the recommended solution is:

RECURSIVE BINARY SEARCH with sorted records.

Reason 1 – Efficient Search

Binary Search requires:

$$\Theta(\log n)$$

time per search after preprocessing.

For a very large sorted database, this is much better than Linear Search:

$$\Theta(n)$$

Reason 2 – Large Dataset

The database can contain 1,000,000+ records.

For:

$$n = 1,000,000$$

Binary Search requires approximately:

$$\log_2(1,000,000) \approx 20$$

levels.

Linear Search may require up to:

$$1,000,000$$

comparisons.

Therefore, Binary Search scales significantly better.

Reason 3 – Database Updates

The original scenario does not indicate that records are continuously modified. If the data can remain sorted and searches are frequent, the initial sorting cost can be amortized over many searches.

Preprocessing Trade-off

The main disadvantage is:

Sorting cost = $\Theta(n \log n)$

However, once the records are sorted, repeated searches are very efficient.

For m searches:

T_{total}

$$= \Theta(n \log n) + \Theta(m \log n)$$

When m becomes large, the one-time sorting cost becomes less significant relative to the large number of efficient searches.

Comparison With Alternatives

Linear Search:

Advantage:

No preprocessing.

Disadvantage:

$\Theta(n)$ search time.

Hash Search:

Advantage:

Expected $\Theta(1)$ lookup.

Disadvantages:

Additional memory, hash-table construction, collision handling and dependency on a suitable hashing strategy.

Binary Search:

Advantage:

Predictable $\Theta(\log n)$ search time with relatively low additional memory.

Therefore, Binary Search provides a strong balance between preprocessing cost, search efficiency, memory requirements and predictable performance.

Final Recommendation

The recommended design is:

SORTED STUDENT DATABASE

↓

RECURSIVE BINARY SEARCH

↓

FAST STUDENT RECORD RETRIEVAL

This solution is practical for large datasets when the database is relatively stable and repeated searches are required.

=====

TASK 12 – DAY-4 DESIGN CHALLENGE

Changed Requirement:

The university now needs to process:

50,000 search requests per hour

on a database containing:

1,000,000 student records.

New student records are added only:

once per day.

The assignment allows the original design to be retained, modified or redesigned.
Design and Analysis of Algorith...

A. IMPACT ANALYSIS – 2 MARKS

The major changes are:

1. The database contains 1,000,000 records.
2. Search frequency is 50,000 searches per hour.
3. New records are added only once per day.

The search frequency is extremely high.

50,000 searches per hour means:

$$50,000 / 60$$

≈ 833.33 searches per minute.

Approximately:

$$833.33 / 60$$

≈ 13.89 searches per second.

Therefore, the system must handle approximately 14 searches per second on average.

The important observation is that searches are frequent while updates are relatively infrequent.

This changes the design decision because a preprocessing cost can now be justified if it significantly reduces the cost of every search.

The low update frequency means that an expensive structure can be constructed once per day and then reused for many searches.

B. EVALUATION OF THE ORIGINAL DESIGN – 2 MARKS

The original recommendation was Binary Search on sorted data.

For $n = 1,000,000$:

$$\log_2(1,000,000) \approx 20.$$

Therefore, one Binary Search requires only approximately 20 levels.

At 50,000 searches per hour, the theoretical number of search levels is approximately:

$$50,000 \times 20$$

$$= 1,000,000 \text{ comparison-level operations per hour.}$$

The initial sorting cost is approximately:

$$\Theta(n \log n)$$

but it only needs to be performed when the data changes.

Since new records are added only once per day, the sorting cost does not need to be repeated after every search.

Therefore, the original Binary Search solution remains practical.

However, Hash Search can provide expected $\Theta(1)$ lookup time and may reduce the search work further.

Thus, the new requirement makes it worthwhile to reconsider whether Binary Search should remain the final design.

C. ADAPTATION OR REDESIGN – 2 MARKS

Decision:

MODIFY THE ORIGINAL DESIGN.

The proposed revised design is:

SORTED DATABASE

+

HASH TABLE INDEX

↓

FAST HASH-BASED SEARCH

The system can maintain a hash index for registration numbers.

When the daily database update occurs:

1. New records are added.
2. The hash index is updated.
3. The updated structure is used for the next day's searches.

Revised Complexity

Daily construction/update:

Expected $\Theta(n)$

Search:

Expected $\Theta(1)$

For m searches:

$$T_{\text{total}} = \Theta(n) + \Theta(m)$$

For:

$$n = 1,000,000$$

and:

$$m = 50,000 \text{ per hour,}$$

the one-time construction cost is acceptable because the resulting structure is reused for a large number of searches.

Therefore, the high search frequency and low update frequency favour a hash-based design.

D. EVIDENCE-BASED DEFENCE – 2 MARKS

The final decision is based on mathematical complexity, search frequency, dataset size, preprocessing cost and practical trade-offs.

Mathematical Evidence

Linear Search:

$$\Theta(n)$$

Binary Search:

$$\Theta(\log n)$$

Hash Search:

$$\text{Expected } \Theta(1)$$

For:

$$n = 1,000,000,$$

Binary Search requires approximately:

$$\log_2(1,000,000) \approx 20$$

search levels.

Linear Search may require up to:

1,000,000

comparisons.

Hash Search requires approximately constant expected lookup work.

Search Frequency Evidence

The system performs:

50,000 searches/hour.

Therefore, even a small improvement in the cost of each search becomes significant over thousands of searches.

Update Frequency Evidence

New records are added only once per day.

This means the system can afford preprocessing because the preprocessing cost can be reused for a very large number of searches.

Dataset Evidence

The database contains:

1,000,000 records.

At this scale, Linear Search is not an appropriate primary search strategy.

Trade-offs

Hash Search requires:

- Additional memory
- Hash-table construction
- Collision handling
- A good hash function

Binary Search requires:

- Sorted records
- Sorting/maintenance cost

However, because the database changes only once per day and searches occur 50,000

times per hour, the benefit of extremely fast expected lookup outweighs the additional preprocessing and memory requirements.

Final Decision

The final recommended design for the changed requirement is:

A hash-indexed student record system with daily update/rebuild of the hash structure.

The overall expected cost is:

$$T_{\text{total}} = \Theta(n) + \Theta(m)$$

where:

n = number of records

m = number of searches.

For repeated searches, this is highly suitable because the one-time preprocessing cost is amortized over a large number of lookups.

If predictable ordered traversal or range-based searches are also required, the sorted database and Binary Search can be retained alongside the hash index.

=====

OVERALL CONCLUSION

The student record retrieval problem demonstrates the importance of selecting an algorithm according to both input size and operating conditions.

Three different approaches were analysed:

1. Linear Search
2. Recursive Binary Search
3. Hash Search

Linear Search is simple and requires no preprocessing, but its $\Theta(n)$ growth makes it unsuitable for very large databases with frequent searches.

Binary Search uses the Divide-and-Conquer strategy. Its recurrence is:

$$T(n) = T(n/2) + c$$

Solving this recurrence gives:

$$T(n) = \Theta(\log n)$$

Therefore, it is highly efficient for searching a sorted database.

Hash Search uses direct access through a hash function. Its expected search complexity is:

$$\Theta(1)$$

although its worst case can become $\Theta(n)$ when severe collisions occur.

For the original problem, Binary Search is a strong choice because it provides predictable logarithmic search performance with reasonable preprocessing.

However, after the requirement changes to 50,000 searches per hour on 1,000,000 records with only one update per day, the system should be modified to use a hash-based index.

The reason is that the high number of searches makes search-time efficiency extremely important, while the low update frequency makes preprocessing acceptable.

Therefore, the final recommended design is:

Original design:

Sorted database + Recursive Binary Search

Revised high-search design:

Hash-indexed database + Expected $O(1)$ lookup

This analysis demonstrates the major CO1 concepts of algorithm design, basic operations, mathematical time-complexity analysis, best/average/worst-case analysis, asymptotic notation, recurrence formulation, recurrence solving, experimental validation and evidence-based algorithm selection.

